

Experiment 2

1. Aim

To design and simulate an **8×1 Multiplexer** (using a direct 8×1 MUX and also by cascading seven 2×1 MUXes), a **Priority Encoder** (8-to-3), and a **Combinational Logic Circuit** using basic gates in **Logisim Evolution**, and to verify their outputs for all valid input combinations.

2. Theory

Multiplexer (MUX): A Multiplexer is a combinational circuit that selects one of many input lines and routes it to a single output line based on select signals. An **8×1 MUX** has 8 data inputs (I0–I7), 3 select lines (S2, S1, S0), and 1 output. The same 8×1 MUX can be realised by cascading **seven 2×1 MUXes** in a tree structure — four at the first level, two at the second, and one at the final level.

Priority Encoder: A Priority Encoder accepts multiple input lines and generates a binary code corresponding to the highest-priority active input. An **8-to-3 Priority Encoder** takes 8 inputs (I0–I7) and produces a 3-bit binary output (Y2 Y1 Y0). Input I7 has the highest priority; I0 has the lowest.

Combinational Logic Circuit: A combinational circuit whose output depends solely on the current input values, with no memory or feedback. It is built using standard logic gates (AND, OR, NOT, XOR) and implements a defined Boolean function.

8×1 MUX — Function Table:

S2	S1	S0	Output (Y)
0	0	0	I0
0	0	1	I1
0	1	0	I2
0	1	1	I3
1	0	0	I4
1	0	1	I5
1	1	0	I6
1	1	1	I7

8-to-3 Priority Encoder — Truth Table (X = don't care):

I7	I6	I5	I4	I3	I2	I1	I0	Y2	Y1	Y0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	X	0	0	1
0	0	0	0	0	1	X	X	0	1	0
0	0	0	0	1	X	X	X	0	1	1
0	0	0	1	X	X	X	X	1	0	0
0	0	1	X	X	X	X	X	1	0	1
0	1	X	X	X	X	X	X	1	1	0
1	X	X	X	X	X	X	X	1	1	1

3. Procedure

1. Open **Logisim Evolution** and create a new project file.
2. **8×1 MUX (direct)**: Place an 8×1 MUX component from the library. Add 8 input pins (I0–I7) and 3 select pins (S2, S1, S0). Connect the single output pin and verify by toggling the select lines.
3. **8×1 MUX using 2×1 MUXes**: Place seven 2×1 MUX components. Wire four MUXes at Level 1 (each receiving two data inputs), two MUXes at Level 2 (receiving Level 1 outputs), and one final MUX at Level 3. Route the common select lines appropriately.
4. **Priority Encoder**: Place an 8-to-3 priority encoder component (Pri). Connect 8 input pins and label the 3-bit output pins Y2, Y1, Y0. Verify that the highest active input produces the correct binary output code.
5. **Combinational Circuit**: Identify the Boolean expression from the given truth table. Implement it using AND, OR, and XOR gates as required. Connect input and output pins, and verify all minterms.
6. Use the **Poke tool** to apply all input combinations to each sub-circuit and record the outputs.
7. Take a screenshot of all completed and verified circuits on the canvas.

4. Circuit Design / Implementation

The following circuits were implemented in **Logisim Evolution**:

- **8×1 MUX (direct)** — Single built-in MUX block with 8 data inputs and 3 select lines, output verified via a probe.
- **8×1 MUX (cascaded 2×1)** — Seven 2×1 MUX blocks arranged in a 3-level binary tree: 4 → 2 → 1, sharing select lines.
- **8-to-3 Priority Encoder** — Built-in Pri component with 8 inputs and a 3-bit encoded output (Y2 Y1 Y0).
- **Combinational Logic Circuit** — Implemented using AND and OR gates to realise a multi-output Boolean function, verified for all input states.

All wires are shown in green (logic-high active state during simulation). Output probes were used to monitor results in real time.

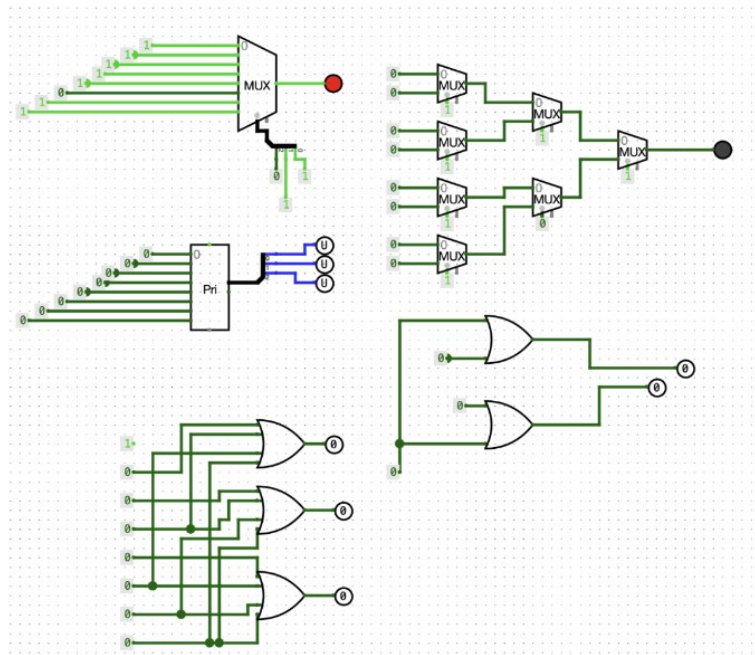


Figure 1: Logisim Evolution canvas showing (top-left) 8×1 MUX, (top-right) 8×1 MUX using seven 2×1 MUXes, (middle-left) 8-to-3 Priority Encoder, and (bottom) Combinational Logic Circuits.

5. Result

All four circuits were successfully designed and simulated in Logisim Evolution. The **8×1 MUX** correctly routed the selected input to the output for all eight select-line combinations. The **cascaded 2×1 MUX** tree produced identical results, confirming functional equivalence. The **8-to-3 Priority Encoder** generated the correct 3-bit code for each highest-priority active input. The **combinational circuit** outputs matched the expected Boolean expressions for all input states.

6. Conclusion

This experiment demonstrated the design and simulation of key combinational circuits — Multiplexers, Priority Encoders, and Boolean logic networks — in Logisim Evolution. The equivalence between a direct 8×1 MUX and a cascaded 2×1 MUX tree highlighted the hierarchical nature of digital circuit design. These building blocks form the foundation of data routing and encoding in modern digital systems.